

# 用 Lean 形式化验证三国杀规则与战术

## Rule Consistency and Strategy Verification

SanguoshaNew 项目汇报

2026 年 8 月 29 日

## 20 分钟汇报结构

- ① 基于 Lean 构建三国杀的基本系统
- ② 三国杀的规则一致性: Rule Consistency
- ③ 三国杀的战术检验: Strategy Verification

### 核心问题

能否把“规则是否自治”和“某条战术是否必然成立”交给 Lean 内核检查?

# 为什么用 Lean 建模三国杀

三国杀规则的难点不在于单条规则复杂，而在于规则之间会相互叠加。

- 牌会在手牌、装备区、判定区、处理区、弃牌堆之间移动。
- 同一时机可能有多个效果或技能等待结算。
- 直觉推演容易漏掉锁定技、虚拟牌、区域移动和先后顺序。
- Lean 的价值是把这些推演写成可检查的定义和定理。

# 项目结构

## SanguoshaNew/

Basic.lean

阶段、时机、区域、玩家、武将、技能

Card.lean

花色、颜色、基础牌、锦囊、武器

State.lean

玩家状态、游戏状态、牌区计数

Rules.lean

合法性判断和状态转移

Invariant.lean

牌数守恒等全局不变量

Judgement.lean

判定、延时锦囊、改判顺序

MaLiangSixDamage.lean

马良六伤路径证明

Examples.lean

小型可执行样例

Exercises.lean

基础证明练习

STUDY\_GUIDE.md

学习路线说明

## 回合与时机

- 阶段：准备、判定、摸牌、出牌、弃牌、结束
- 时机：空闲、响应、结算、阶段开始、阶段结束

## 区域与玩家

- 区域：手牌、装备区、判定区、处理区、弃牌堆
- 当前主模型先使用两人局状态
- 特定战术证明可建立专用多人最小状态

# 牌与武将系统

## 牌名系统

基础牌已经区分【杀】【火杀】【雷杀】【闪】【桃】【酒】；证明需要时进一步登记【顺手牵羊】【借刀杀人】【丈八蛇矛】。

## 武将与技能

当前模型登记了张角、司马懿、马良、郝昭、神郭嘉：

General.maLiang -> [Skill.zishu, Skill.yingyuan]

General.haoZhao -> [Skill.zhengu]

General.shenGuojia ->  
[Skill.shenHuishi, Skill.tianyi, Skill.huiShiFarewell]

# 状态转移接口

项目的主规则接口是：

```
transition : GameState -> Command -> Option GameState
```

- 输入：当前局面和玩家命令。
- 输出：合法则返回 `some` 新局面，非法则返回 `none`。
- 好处：非法操作不会被悄悄当成正常操作继续结算。

## 已经实现的基础行动

使用【杀】、响应【闪】、使用【桃】、使用【酒】、结算待处理【杀】、结束阶段。

# 本轮更新

这次我们把项目往“学员可读、可练、可验证”的方向再推了一步：

- 新增 `Exercises.lean`, 把 `legal`、`transition`、`healOne` 等基础证明拆成练习题。
- 扩充 `Examples.lean`, 补进非法命令返回 `none` 和 `endPhase` 的可运行样例。
- 在 `Rules.lean` 里补了 `transition_eq_none_iff_not_legal` 和 `legal_of_transition_some`。
- 在 `State.lean` 里补齐了牌池总数、玩家牌数守恒等基础引理。
- 新增 `STUDY_GUIDE.md`, 单独给出阅读顺序和练习路径。



# 规则一致性问题

## Research Question

Given a formalized game, can Lean verify that its rules preserve global invariants and that ambiguities in the ordering of game effects are semantically harmless?

在本项目中，这个问题被拆成两类检查：

- 全局不变量：规则执行后，牌不会凭空产生或消失。
- 顺序歧义：若两个效果顺序不同，最终语义是否真的相同。

# 牌数守恒

牌区模型使用计数结构 CardPool:

```
def totalCards (s : GameState) : Nat :=  
  s.playerA.cardTotal  
  + s.playerB.cardTotal  
  + s.drawPile.total  
  + s.discardPile.total  
  + s.processing.total
```

规则执行通常只是移动牌的位置，例如：

- 使用【杀】：手牌减少，处理区增加。
- 使用【闪】：手牌减少，弃牌堆增加。
- 结算结束：处理区进入弃牌堆。

# 牌数守恒的 Lean 定理

总定理:

```
theorem transition_preserves_totalCards
  (s : GameState)
  (c : Command)
  (s' : GameState)
  (htrans : transition s c = some s') :
  s'.totalCards = s.totalCards
```

- 前提: 命令  $c$  成功执行, 得到新局面  $s'$ 。
- 结论: 新旧局面的总牌数相等。
- 证明方法: 按命令分类, 分别证明每种规则不会破坏全局牌数。

## 顺序歧义：张角与司马懿改判

经典争议：张角和司马懿同时在场时，判定牌到底谁先改、谁后改？

- 张角【鬼道】：可以用一张黑色牌替换当前判定牌。
- 司马懿【鬼才】：可以用一张手牌替换当前判定牌。
- 改判本质上是“后执行者覆盖当前判定牌”。

### Lean 建模

将改判动作写成列表，按列表顺序依次覆盖当前判定牌。

## 张角-司马懿实例

同一局面，同样两次合法改判，只交换顺序：

```
resolveJudgement initialJudgementCard  
    [simaGuicaiToHeart, zhangGuidaoToSpade]
```

```
resolveJudgement initialJudgementCard  
    [zhangGuidaoToSpade, simaGuicaiToHeart]
```

- 司马懿先改、张角后改：最终为黑桃 2，【闪电】命中。
- 张角先改、司马懿后改：最终为红桃 K，【闪电】不命中。

# 顺序确实影响游戏结果

核心定理:

```
theorem judgement_replacement_order_affects_game_result :  
  stateAfterSimaThenZhang != stateAfterZhangThenSima
```

## 结论

这个例子证明：改判顺序不是语义无害的实现细节，而会改变最终游戏状态。

- 若最终是黑桃 2，张角被【闪电】造成 3 点伤害。
- 若最终是红桃 K，【闪电】不造成伤害。

# 战术检验问题

## Research Question

Given a game state, can Lean formally verify the existence of a winning sequence, or even a strategy that guarantees victory against every legal response of the opponent?

本项目当前完成的是第一步：

- 给定一个具体局面；
- 形式化一条候选行动序列；
- 证明它对任意未知牌都能造成足够伤害。

# 马良六伤实例：直觉与目标

初始局面：

- 马良装备【丈八蛇矛】。
- 手牌：【酒】、两张【杀】、【顺手牵羊】、【借刀杀人】。
- 马良拥有锁定技【自书】和主动技【应援】。
- 场上有队友郝昭。
- 对手神郭嘉体力 5，手牌为 0，无防御干预。

需要证明

看似五张牌最多四伤，但存在一条总伤害至少为 6 的行动序列。



## 五步六伤路径

- ① 使用【酒】，令下一张【杀】伤害 +1。
- ② 发动【丈八蛇矛】，两张【杀】当一张【杀】；造成 2 伤，并通过【应援】交给郝昭。
- ③ 使用【借刀杀人】，令郝昭出【杀】；造成 2 伤，并通过【应援】把【借刀杀人】交给神郭嘉。
- ④ 使用【顺手牵羊】拿回【借刀杀人】；【自书】锁定触发，摸一张未知牌 x。
- ⑤ 再次使用【借刀杀人】，令郝昭再出【杀】；造成最后 2 伤。

总伤害：2 + 2 + 2 = 6。

## 关键：未知牌 $x$ 不影响路径

第四步之后，马良的手牌结构包含：

- 通过【顺手牵羊】拿回的【借刀杀人】；
- 通过【自书】摸到的一张未知牌  $x$ 。

### 数量不变量

第二次【借刀杀人】只要求马良持有【借刀杀人】，不要求  $x$  是某种特定牌。

因此，这条路径不依赖运气，只依赖【自书】带来的手牌数量保证。

## 任意未知牌的 Lean 表述

未知牌被建模为变量：

`drawn : SixDamageCard`

核心定理使用全称量词覆盖所有可能的 `drawn`：

```
theorem maliang_has_six_damage_path_for_any_unknown_card :  
  forall drawn : SixDamageCard,  
    exists actions : List SixDamageAction,  
      actions = sixDamageActions drawn /\  
        (runSixDamagePath drawn).totalDamageToShen >= 6 /\  
        (runSixDamagePath drawn).shenGuojiaHp = 0
```

# 证明结构

证明中有两个关键层次：

```
theorem after_step_four_has_borrowed_sword_for_any_drawn
  (drawn : SixDamageCard) :
  (stateAfterStepFour drawn).maLiangHand.borrowedSword >= 1
```

```
theorem run_six_damage_path_total_damage_eq_six_for_any_drawn
  (drawn : SixDamageCard) :
  (runSixDamagePath drawn).totalDamageToShen = 6
```

- `simp` 处理与未知牌无关的逻辑展开。
- `omega` 处理总伤害  $\geq 6$  这样的数值不等式。
- Lean 内核通过后，结论对所有可能的  $x$  成立。

# 总结

- ① 基本系统：用 Lean 明确定义阶段、时机、区域、牌、武将、技能和状态转移。
- ② Rule Consistency：用定理检查规则是否保持全局不变量，并识别顺序歧义是否语义有害。
- ③ Strategy Verification：用存在性证明和全称量化验证战术路径不依赖直觉或运气。

## 下一步

将当前的专用证明状态推广为通用多人 GameState，并把响应窗口、装备区合法性和完整牌区移动纳入统一规则引擎。

谢谢